

Full Stack Development with MERN Project

Documentation format

1. Introduction

- **Project Title:** SHOPEZ E-commerce Application
- **Team Member:** BENAKALA PRANITHA

2. Project Overview

Purpose:

- The primary purpose of SHOPEZ is to provide an efficient online shopping platform where customers can purchase products without visiting a physical store. The application offers a simple interface that allows users to register, log in securely, browse products, search for desired items, add products to their cart or wishlist, and place orders.
- From a technical perspective, the project demonstrates how a complete full-stack web application can be developed using modern JavaScript technologies. It integrates frontend, backend, and database technologies to provide a seamless shopping experience.
- The project also helps in understanding authentication mechanisms, CRUD operations, API development, database interactions, and deployment strategies.

Objectives

- Develop a secure online shopping platform.
- Provide an attractive and responsive user interface.
- Implement user authentication using JWT.
- Store product and user information securely in MongoDB Atlas.
- Allow users to manage carts and wishlists efficiently.
- Enable customers to place and view orders.
- Demonstrate full-stack development using the MERN stack.

Features:

- User Registration
- Secure Login & Logout
- JWT Authentication
- Product Listing
- Product Search
- Product Details
- Shopping Cart
- Wishlist
- Order Placement
- Order History

- Admin Dashboard
- Responsive Design
- RESTful API Integration
- MongoDB Cloud Database

3. Architecture

Frontend:

- The frontend is developed using React.js, a component-based JavaScript library that enables the creation of reusable UI components. React Router is used for client-side navigation between pages such as Home, Login, Register, Product Details, Cart, Wishlist, and Orders.
- React Hooks such as useState and useEffect are used for state management and lifecycle handling. Axios is used to send HTTP requests to the backend APIs.

Advantages of the React frontend include:

- Component reusability
- Faster rendering using Virtual DOM
- Better user experience
- Responsive interface
- Easy maintenance

Backend

The backend is developed using Node.js and Express.js.

Express.js provides RESTful APIs that communicate with the frontend. The backend handles:

- User Registration
- User Login
- JWT Authentication
- Product APIs
- Cart APIs
- Wishlist APIs
- Order APIs

Middleware is used for:

- Authentication
- Error Handling
- JSON Parsing
- CORS Configuration

Passwords are encrypted using bcrypt before being stored in the database.

Database:

Database Architecture

MongoDB Atlas is used as the cloud-based NoSQL database.

Collections include:

Users

Stores

- Name
- Email
- Password (Encrypted)

Products

Stores

- Product Name
- Description
- Price
- Image
- Category

Wishlist

Stores

- User ID
- Product ID

Cart

Stores

- User ID
- Product ID
- Quantity

Orders

Stores

- User ID

- Ordered Products
- Total Amount
- Order Date

Mongoose is used for defining schemas and performing CRUD operations.

4. Setup Instructions

Software Requirements

- Node.js
- npm
- MongoDB Atlas Account
- Git
- Visual Studio Code

Installation Steps

Step 1

- Clone the repository
- git clone <https://github.com/PranithaBenakala41/ShopEZ.git>

Step 2

- Open the project
- cd ShopEZ

Step 3

- Install frontend dependencies
- cd client
npm install

Step 4

- Install backend dependencies
- cd ../server
npm install

Step 5

Create .env

- PORT=5000
- JWT_SECRET= mysecretkey
- MONGO_URI= mongodb://benakala_pranitha_41:PraniBabe41@ac-zbwolmx-shard-00-00.9ya1w60.mongodb.net:27017,ac-zbwolmx-shard-00-01.9ya1w60.mongodb.net:27017,ac-zbwolmx-shard-00-02.9ya1w60.mongodb.net:27017/ShopEZ?ssl=true&replicaSet=atlas-7c0k8v-shard-0&authSource=admin&appName=Cluster0

Step 6

- Run Backend
- npm start

Step 7

- Run Frontend

- npm start

5. Folder Structure

ShopEZ/

client/

```
|
|
|— src/
|   |— components/
|   |— pages/
|   |— App.js
|   |— index.js
|
|— public/
```

server/

```
|
|— models/
|— routes/
|— controllers/
|— middleware/
|— config/
|— server.js
|— package.json
|— README.md
```

6. Running the Application

Provide commands to start the frontend and backend servers locally.

Frontend: `npm start` in the client directory.

Backend: `npm start` in the server directory.

7. API Documentation

User APIs

Register

- **POST** `/api/auth/register`
- Registers a new user.

Login

- **POST** `/api/auth/login`
- Returns JWT token after successful authentication.

Product APIs

- **GET** `/api/products`
- Returns all products.
- **GET** `/api/products/:id`
- Returns product details.

Cart APIs

- **POST** `/api/cart`
- Adds products to cart.
- **DELETE** `/api/cart/:id`
- Removes products.

Wishlist APIs

- **POST** `/api/wishlist`
- Adds products.
- **DELETE** `/api/wishlist/:id`
- Removes products.

Order APIs

- **POST** `/api/orders`

- Places an order.
- **GET /api/orders**
- Returns previous orders.

8. Authentication

Authentication is implemented using JSON Web Tokens (JWT).

Workflow:

- User logs in.
- Backend verifies credentials.
- JWT token is generated.
- Token is stored in Local Storage.
- Token is sent with every protected request.
- Backend validates the token before allowing access.
- Passwords are encrypted using bcrypt before storage.

9. User Interface

The application consists of the following pages:

- Home Page
- Login Page
- Registration Page
- Product Details
- Shopping Cart
- Wishlist
- Orders
- Navigation Bar

The UI is responsive and adapts to desktops, tablets, and mobile devices.

10. Testing

The application was tested using manual functional testing.

The following modules were tested:

- Registration
- Login
- Search
- Product Details
- Wishlist
- Cart
- Orders
- Logout

Positive and negative test cases were executed successfully.

11.Screenshots Link:

<https://github.com/PranithaBenakala41/ShopEZ/tree/main/screenshots>

12.Known Issues

- No integrated online payment gateway.
- Product reviews and ratings are unavailable.
- Email notifications are not implemented.

13. Future Enhancements

Future Enhancements

Future versions of SHOPEZ can include:

- Online payment integration (Stripe/Razorpay)
- AI-based product recommendations
- Product reviews and ratings
- Order tracking
- Email notifications
- Inventory management
- Coupons and discount system

- Multi-language support
- Dark mode
- Push notifications
- Chatbot-based customer support